Practical 1 Report

Graded

22 Hours, 44 Minutes Late

Group

Samson Okuthe
Nyakallo Peete
View or edit group

Total Points

40.75 / 60 pts

Question 1

Introduction

4.25 / 5 pts

Problem Statement

+ 2 pts **Excellent**
Clearly states the aim of the practical; explains relevance of performance comparison; provides context for edge detection and Sobel operator; introduces benchmarking concept; establishes clear objectives

✔ + 1.5 pts **Good**
States practical aim clearly; some context for performance comparison; mentions Sobel operator; mostly establishes objectives

+ 1 pt **Satisfactory**
States basic aim; minimal context provided; may lack clarity on importance; objectives not fully articulated

+ 0 pts **Unsatisfactory**
Missing or poorly explained problem statement; no clear context

---

Implementation Strategy Overview

✔ + 2 pts **Excellent**
Describes manual loop-based approach; explains built-in conv2() function; mentions padding strategy (full/same/valid); previews performance metrics; clear comparison outline

+ 1.5 pts **Good**
Describes both approaches; mentions padding strategy; indicates what will be compared; some details could be more specific

+ 1 pt **Satisfactory**
Basic description of implementations; minimal detail on approach; vague on comparison methodology

+ 0 pts **Unsatisfactory**
Missing or incoherent strategy description

Benchmarking Goals

+ 1 pt **Excellent**
Clearly states goal to measure execution times; specifies speedup ratio calculation; mentions correctness verification; indicates analysis across image sizes

✔ + 0.75 pts **Good**
States goal to compare performance; mentions speedup calculation; some benchmarking goals outlined

+ 0.5 pts **Satisfactory**
Vague benchmarking goals; incomplete statement of objectives

+ 0 pts **Unsatisfactory**
No clear benchmarking goals stated

1 Too long

Question 2

Methodology

8.5 / 10 pts

Manual Implementation Approach

+ 3 pts Excellent
Explains nested loop structure for convolution; describes kernel application; explains padding/boundary handling method; justifies padding choice; discusses Gx and Gy kernels; explains gradient magnitude calculation

✔ **+ 2.5 pts Good**
Explains nested loop approach; describes kernel application; mentions padding; explains Gx/Gy application; minor details missing

+ 2 pts Satisfactory
Basic explanation of nested loops; mentions padding and kernel application; some technical details unclear; lacks justification

+ 1 pt Needs Improvement
Vague or incomplete explanation; missing key details like padding strategy

+ 0 pts Unsatisfactory
Missing or fundamentally incorrect explanation

Built-in Function Implementation

+ 2 pts Excellent
Explains conv2() function; specifies mode (full/same/valid) and justifies choice; justifies consistency with manual; notes preprocessing/postprocessing; explains Gx and Gy application

✔ **+ 1.5 pts Good**
Explains conv2() usage; specifies mode choice; indicates consistency; minor details lacking

+ 1 pt Satisfactory
Basic explanation of conv2(); mode mentioned but justification weak; incomplete description

+ 0 pts Unsatisfactory
Missing or incorrect explanation

Timing and Measurement Methodology

✔ **+ 3 pts Excellent**
Clearly describes tic/toc or timeit() usage; explains number of runs for accuracy; describes averaging method; addresses warm-up effects; explains speedup formula; justifies choices; mentions all 5 test images

+ 2.5 pts Good
Describes timing methods clearly; mentions multiple runs; explains averaging; speedup explained; some methodological choices justified

+ 2 pts Satisfactory
Describes tic/toc usage; mentions multiple runs with minimal detail; speedup explained simply; limited justification

+ 1 pt Needs Improvement
Vague timing methodology; unclear speedup calculation

+ 0 pts Unsatisfactory
Missing timing methodology

Function Signature and Input Handling

+ 2 pts Excellent
Describes output comparison method; explains tolerance/threshold for comparison; mentions comparison

functions; discusses verification across all images; explains handling of numerical differences

✔ **+ 1.5 pts** **Good**
Describes output comparison; mentions tolerance for numerical differences; indicates verification across images

+ 1 pt **Satisfactory**
Basic comparison description; tolerance mentioned but not detailed; incomplete verification

+ 0 pts **Unsatisfactory**
No verification methodology described

Question 3

Results

7.5 / 10 pts

Data Table or Summary

+ 4 pts Excellent
Clear, well-formatted table for all 5 images; columns: Image size, T_manual, T_builtin, Speedup; logically organized; units clearly specified; accurate data; clear caption; no missing points

✔ **+ 3 pts Good**
Clear table with most required info; results for all 5 images; mostly accurate; minor formatting/clarity issues

+ 2 pts Satisfactory
Table present with basic data; may be missing some images; formatting improvable; some units unclear

+ 1 pt Needs Improvement
Incomplete table; missing significant data; poor organization

+ 0 pts Unsatisfactory
No table or data summary

Graphical Representation

+ 4 pts Excellent
High-quality plot(s) showing comparison; plots: bar chart or line graph; axes clearly labeled with units; legend present and clear; descriptive title; professional formatting; appropriate colors/fonts; clearly communicates findings

✔ **+ 3 pts Good**
Clear, informative plot showing results; axes labeled, legend present; professional appearance; could be improved in one aspect; effectively communicates results

+ 2 pts Satisfactory
Plot present showing results; basic labeling; some aspects unclear (legend, title, labels); communicates results but could be clearer

+ 1 pt Needs Improvement
Plot present but poorly formatted; missing critical labels; hard to interpret

+ 0 pts Unsatisfactory
No graphs or plots shown

Statistical Summary

+ 2 pts Excellent
Reports mean, min, max speedup; indicates scaling with image size; includes variability measure; discusses consistent performance; identifies outliers

✔ **+ 1.5 pts Good**
Provides mean speedup and variability; discusses trend; some statistical context

+ 1 pt Satisfactory
Provides basic speedup statistics; limited analysis of trend

+ 0 pts Unsatisfactory
No statistical summary

Question 4

Discussion

17 / 25 pts

Performance Analysis

+ 6 pts Excellent
Clear explanation of why one is faster; discusses MATLAB optimization; explains loop overhead; addresses memory access patterns if relevant; discusses implementation trade-offs; quantifies gap

✔ **+ 5 pts Good**
Clear analysis of performance differences; explains optimization and built-in advantages; mentions loop overhead; thorough but could go deeper

+ 4 pts Satisfactory
Identifies performance differences; attempts explanation; some superficial analysis; could be more detailed

+ 2 pts Needs Improvement
Weak analysis; limited or unclear explanations

+ 0 pts Unsatisfactory
Missing or incoherent analysis

Scalability Analysis

+ 5 pts Excellent
Analyzes how both scale with image size; discusses complexity ($O(n \cdot m \cdot k^2)$); explains degradation; HPES implications; optimization opportunities; which scales better; evidence from data

+ 4 pts Good
Discusses how both implementations scale; mentions complexity or trend; embedded systems context; size-dependent behavior analysis

✔ **+ 3 pts Satisfactory**
Observes scalability trend from data; limited analysis of why; basic discussion of implications

+ 1 pt Needs Improvement
Weak or unclear scalability analysis; minimal connections

+ 0 pts Unsatisfactory
No scalability discussion

Correctness Verification Discussion

+ 4 pts Excellent
Confirms identical/numerically equivalent results; explains numerical differences; discusses tolerance thresholds; addresses verification importance; mentions edge cases

+ 3 pts Good
Confirms correctness; mentions numerical precision; discusses verification importance

✔ **+ 2 pts Satisfactory**
States both are correct; minimal detail on verification; limited precision discussion

+ 0 pts Unsatisfactory
No verification discussion or suggests incorrect results

Implementation Trade-offs

+ 5 pts Excellent
Discusses manual control vs. built-in optimization; code clarity vs. performance; when each is appropriate; development vs. runtime time; optimization opportunities; custom requirements; high performance embedded systems context

+ 4 pts **Good**
Discusses several trade-offs; code clarity vs. performance; considers real-world applications; good analysis

✔ **+ 3 pts** **Satisfactory**
Identifies some trade-offs; limited depth or relevance; could be more comprehensive

+ 1 pt **Needs Improvement**
Weak discussion of trade-offs; limited context

+ 0 pts **Unsatisfactory**
No trade-off discussion

Critical Analysis and Insights

+ 5 pts **Excellent**
Thoughtful analysis beyond numbers; identifies surprising results; discusses limitations (small images, overhead dominance); suggests further investigation; embedded systems connections; practical significance; real-world applicability

✔ **+ 4 pts** **Good**
Good critical analysis; identifies limitations; discusses implications; thoughtful insights

+ 3 pts **Satisfactory**
Some critical analysis; mentions limitations; limited depth

+ 1 pt **Needs Improvement**
Superficial analysis; missing critical perspective

+ 0 pts **Unsatisfactory**
No critical analysis

Question 5

Conclusion

3.5 / 5 pts

Summary of Findings

+ 2 pts Excellent
Concisely summarizes key findings (which is faster, by how much); restates main metrics; recaps comparison across sizes; clear takeaway

✔ **+ 1.5 pts Good**
Good summary of findings; mentions key metrics; clear message

+ 1 pt Satisfactory
Basic summary of results; some key points covered

+ 0 pts Unsatisfactory
Missing or unclear summary

Limitations and Future Work

+ 2 pts Excellent
Identifies limitations (hardware-specific, MATLAB version, kernel size, no parallelization); suggests specific future work (parallel, GPU, optimized loops); discusses hardware differences; potential optimization

✔ **+ 1.5 pts Good**
Identifies some limitations; suggests areas for future work; some relevant suggestions

+ 1 pt Satisfactory
Mentions limitations or future work; limited detail

+ 0 pts Unsatisfactory
No limitations or future work discussed

Final Implications

+ 1 pt Excellent
Clearly states practical implications for high performance embedded systems; explains when to use each; actionable guidance

+ 0.75 pts Good
Mentions implications for high performance embedded systems; some practical guidance

✔ **+ 0.5 pts Satisfactory**
Brief mention of implications

+ 0 pts Unsatisfactory
No implications discussed

Question 6

Formatting and Styling

0 / 5 pts

IEEE Conference Format Compliance

+ 2 pts **Excellent**
Follows IEEE format strictly; proper section numbering; correct dimensions/margins; professional layout; proper spacing; correct citation format; appropriate headers/footers

✔ + 1.5 pts **Good**
Mostly follows IEEE format; minor deviations; generally professional

+ 1 pt **Satisfactory**
Attempts IEEE format; some inconsistencies; readable but not fully compliant

+ 0 pts **Unsatisfactory**
Does not follow IEEE format

Length and Content Distribution

+ 1.5 pts **Excellent**
Report is ≤ 2 pages; well-distributed content; no excessive white space; sections appropriately sized; effective use of space for tables/figures

✔ + 1 pt **Good**
Meets 2-page limit; good distribution; minor spacing issues

+ 0.75 pts **Satisfactory**
Meets limit but distribution could be better; some crowding or sparse areas

+ 0 pts **Unsatisfactory**
Exceeds page limit or very poor distribution

Writing Quality and Clarity

+ 1.5 pts **Excellent**
Clear professional scientific writing; proper grammar/spelling; appropriate terminology; concise sentences; logical flow; proper tense; figures well-captioned and referenced

✔ + 1 pt **Good**
Clear writing with minor grammatical issues; good terminology; generally well-structured; most figures referenced

+ 0.75 pts **Satisfactory**
Readable with some unclear passages; minor grammar errors; some terminology issues; some figures lack captions

+ 0 pts **Unsatisfactory**
Poor writing quality, hard to follow

– 8.5 pts Late Submission(-6) and Admin(-2.5) for adding group member to submission on your behalf

Questions assigned to the following page: 1, 2, and 3

Practical 1: Performance Benchmarking of 2D Convolution for Sobel Edge Detection

Samson Okuthe
Department of Electrical Engineering
University of Cape Town
GitHub Repository

Nyakallo Peete
Department of Electrical Engineering
University of Cape Town
GitHub Repository

***Abstract*—This report evaluates the computational performance of algorithmic implementations in image processing by comparing a manually coded 2D convolution against MATLAB's optimized built-in `conv2` function. Using Sobel operators for edge detection, execution times were benchmarked across five image resolutions ranging from 16,384 to 4,194,304 pixels. The results verified mathematical equivalence between both methods. Performance evaluation showed that while both implementations scale with the number of pixels processed for a fixed 3×3 kernel, the compiled and optimized `conv2` implementation achieved substantial speedup. Additional repeated-run experiments reveal that cold-start measurements can be heavily distorted by Just-In-Time (JIT) compilation, library initialization, and runtime overhead, especially for small matrices. Steady-state benchmarking demonstrates consistent acceleration for medium and large image sizes, with remaining variance attributable to system-level effects such as OS scheduling and dynamic CPU frequency scaling.**

***Index Terms*—Sobel edge detection, 2D convolution, MATLAB, performance benchmarking, image processing, cache effects**

I. Introduction

This practical explores computational performance by benchmarking a basic algorithmic implementation against an optimized library-based solution. Specifically, a manual 2D convolution implemented using strictly nested loops is compared to MATLAB's built-in `conv2` function for Sobel edge detection. Performance is evaluated across five image sizes to characterize how execution time scales with increasing input size, and to observe how system-level effects influence measured wall-clock time.

The Sobel operator estimates image gradients in two orthogonal directions, highlighting regions of high spatial frequency associated with edges. Implementing Sobel filtering requires repeated 2D convolution, which provides a clear demonstration of the cost of unoptimized scalar processing in an interpreted environment and the benefits of highly optimized numerical libraries.

II. Methodology

The project was developed in MATLAB to ensure a controlled environment for correctness verification and benchmarking.

A. Data Acquisition and Edge Handling

Five sample images of increasing resolution were pre-loaded into a cell array to decouple disk I/O latency from measured processing time. To ensure robustness, an edge-case handler was implemented to detect 3-channel RGB images and convert them to 1-channel grayscale. Images were cast to `double` precision to avoid arithmetic overflow during convolution and gradient operations. Sobel operators G_x and G_y were defined as 3×3 kernels to approximate horizontal and vertical gradient components.

B. Algorithmic Implementation

Two convolution methods were implemented:

- **Manual Implementation (`my_conv2`):** A custom function using strictly nested `for` loops. The kernel was rotated by $180°$ to implement true convolution (not correlation). Manual zero-padding was applied to preserve input dimensionality and produce a `'same'`-sized output.
- **Built-in Implementation (`inbuilt_conv2`):** A wrapper around MATLAB's `conv2` using the `'same'` option for consistent output shape, enabling a fair comparison with the manual method [1].

C. Benchmarking Strategy and Verification

Benchmarking was performed using `tic`/`toc` wall-clock timing. Two complementary timing viewpoints were used:

- **Cold-start (First-run) timing:** Captures initialization overhead such as JIT compilation, library loading, and runtime setup. These effects can dominate timing for small arrays.
- **Steady-state timing (Repeated runs):** Subsequent runs after initialization provide a clearer measurement of computational throughput. Remaining variance is influenced by OS scheduling noise, background activity, cache state, and dynamic frequency scaling. This aligns with benchmarking cautions discussed in course material [2].

Correctness was validated by comparing manual and built-in outputs using the maximum absolute pixel-wise difference threshold of $< 1 \times 10^{-6}$.

III. Results

All experiments passed verification across all image sizes, confirming that the manual convolution implementation is mathematically equivalent to MATLAB's built-in method under the selected boundary handling.

Questions assigned to the following page: 3, 4, and 5

Table I presents representative steady-state results (3rd run), where most cold-start overhead has been removed. Table II presents the first-run results, which include initialization overhead and show strong distortion for the smallest image size. Table III presents the 5th run, illustrating that runtime variance persists even after warm-up.

TABLE I
BENCHMARKING RESULTS (3RD RUN: STEADY-STATE BEHAVIOR)

Image Size	Total Pixels	Manual (s)	Built-in (s)	Speedup
128×128	16,384	0.0024	0.0002	13.68x
256×256	65,536	0.0074	0.0003	23.54x
512×512	262,144	0.0294	0.0021	13.91x
1024×1024	1,048,576	0.1327	0.0100	13.25x
2048×2048	4,194,304	0.4638	0.0306	15.16x

TABLE II
BENCHMARKING RESULTS (1ST RUN: COLD START / INITIALIZATION INCLUDED)

Image Size	Total Pixels	Manual (s)	Built-in (s)	Speedup
128×128	16,384	0.0114	0.0116	0.98x
256×256	65,536	0.0247	0.0008	29.91x
512×512	262,144	0.0384	0.0024	15.96x
1024×1024	1,048,576	0.0919	0.0075	12.32x
2048×2048	4,194,304	0.3848	0.0317	12.15x

TABLE III
BENCHMARKING RESULTS (5TH RUN: RUNTIME VARIANCE OBSERVED)

Image Size	Total Pixels	Manual (s)	Built-in (s)	Speedup
128×128	16,384	0.0016	0.0002	6.77x
256×256	65,536	0.0062	0.0004	16.30x
512×512	262,144	0.0298	0.0019	15.95x
1024×1024	1,048,576	0.1042	0.0105	9.88x
2048×2048	4,194,304	0.4088	0.0290	14.12x

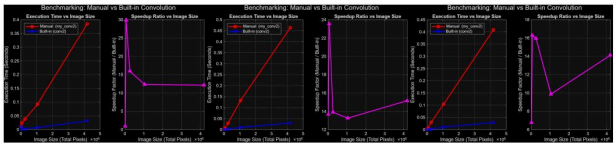

Fig. 1. Combined plot of execution time and speedup ratio from cold-start (1st run) through 5th run. Placeholder image: `Warmupto5th.png`.

IV. DISCUSSION

The benchmarking results demonstrate both algorithmic performance differences and system-level measurement effects.

A. Scaling Behavior and Constant Factors

For a fixed 3×3 Sobel kernel, both implementations scale proportionally with the number of pixels processed (i.e., linear in total pixels). However, the constant factors differ substantially. The manual method executes convolution using MATLAB-level loops, which incur interpreter overhead (loop control, indexing, and runtime checks). In contrast, `conv2` executes in optimized compiled code and is designed for efficient numerical throughput [1]. This difference explains the consistent order-of-magnitude speedups observed in the steady-state regime (Table I).

B. Cold-Start Distortion and Initialization Overhead

The first-run results (Table II) reveal a strong cold-start penalty, most clearly visible for the smallest image. For 128×128, the built-in function appears to provide little or no benefit (0.98x) because initialization overhead dominates the total measured time. Practical sources of this overhead include JIT compilation, library loading, runtime setup, and potential internal thread pool initialization. This confirms the importance of separating initialization effects from steady-state compute performance when benchmarking [2].

C. Steady-State Variance and System-Level Effects

While steady-state runs remove most initialization overhead, Table III shows that repeated runs can still vary, particularly for larger images. Such variance can be attributed to OS scheduling, background activity, cache state differences between runs, and dynamic CPU frequency scaling. These effects motivate statistical benchmarking approaches (e.g., repeated trials and median/mean aggregation) rather than relying on a single run. This aligns with broader perspectives on performance measurement and the need for accurate evaluation facilities in parallel computing systems [3].

D. Interpretation of Speedup Trends

Across medium and large image sizes, `conv2` consistently outperforms manual convolution by roughly one order of magnitude. Differences in speedup across sizes are expected: as images grow, memory traffic increases and the computation becomes increasingly constrained by data movement through the memory hierarchy. When memory bandwidth becomes a dominant factor, the relative advantage of arithmetic optimizations may reduce, even though the built-in method remains faster.

V. CONCLUSION

This practical successfully demonstrated the performance benefits of using optimized library functions for convolution-heavy image processing. Verification confirmed that the manual implementation was mathematically correct. Benchmarking results show that MATLAB's optimized `conv2` provides substantial speedup under steady-state conditions, while first-run timing can be misleading due to initialization overhead. The repeated-run analysis highlights that robust benchmarking should account for cold-start effects and runtime variance to provide representative performance estimates.

VI. AI CLAUSE

An AI language model was used to assist with interactive drafting and refinement of the report structure, benchmarking interpretation, and clarity of explanations regarding convolution scaling, steady-state measurement, and initialization overhead. All code and experimental results were implemented and validated by the authors on local hardware.

Question assigned to the following page: 6

REFERENCES

[1] MathWorks, "`conv2`: 2-D convolution," MATLAB Documentation. [Online]. Available: https://www.mathworks.com/help/matlab/ref/conv2.html. Accessed: Feb. 2026.

[2] S. Winberg, "Lecture 5: Performance Benchmarking & Wall Clock Timing," EEE4120F High Performance Embedded Systems, University of Cape Town, Lecture slides, 2026.

[3] S. Winberg, "Seminar 1: The Landscape of Parallel Computing (Research: A View from Berkeley)," EEE4120F High Performance Embedded Systems, University of Cape Town, Seminar slides, 2026.

APPENDIX A
AGGREGATED MEDIAN BENCHMARKING RESULTS

To provide a compact and statistically representative summary under length constraints, an aggregated steady-state median table is included here. Cold-start measurements were excluded to avoid initialization bias. The medians below were computed using the warm runs available (3rd and 5th runs), which reduces sensitivity to outliers caused by OS scheduling and dynamic frequency scaling. The median speedup is computed as the ratio of the median manual time to the median built-in time.

TABLE IV
AGGREGATED STEADY-STATE MEDIAN RESULTS (COMPUTED FROM WARM RUNS)

Image Size	Total Pixels	Manual Median (s)	Built-in Median (s)	Median Speedup
128×128	16,384	0.0020	0.000 20	10.00x
256×256	65,536	0.0068	0.000 35	19.43x
512×512	262,144	0.0296	0.002 00	14.80x
1024×1024	1,048,576	0.1185	0.010 25	11.56x
2048×2048	4,194,304	0.4363	0.029 80	14.64x